LATVIJAS UNIVERSITĀTE | EKSAKTO ZINĀTŅU UN TEHNOLOĢIJU FAKULTĀTE

Finansē Eiropas Savienība NextGenerationEU

2027 Nacionālais attīstības plāns

Assignment #2

Text classification

Deadline: 30.04.2026

Fundamentals of NLP, DatZB022
Spring semester 2026

#LanguageTechnologyInitiative

Create and evaluate a text classifier

- **Dataset**: acquisition, preprocessing, analysis (max 6 points)
  1. Download the **GoEmotions** dataset from: https://github.com/google-research/google-research/tree/master/goemotions `./data/{train,dev,test}.tsv` (ignore the 3rd column in the TSV files)
  2. **Convert** the original train/dev/test files that use **27** fine-grained emotions (+neutral) **into** train/dev/test files that use only **6** classes of basic emotions (+neutral) – see `./data/ekman_mapping.json`
  3. Note that about **15%** of samples in the dataset are labelled with more than one emotion – remove such samples from the resulting train/dev/test files
  4. Do basic text **tokenization** of your choice (e.g. regex, NLTK, spaCy, Stanza)
  5. Conduct shallow **data analysis** (e.g. samples per class; unigram/bigram TOP frequencies per class; potential impact of frequency-based vocabulary pruning – vocabulary size)
  6. Consider basic text **clean-up** or **normalization** (e.g. numbers=>placeholders, wordforms=>lemmas)
- **Training:** a one-of multinomial Naïve Bayes classifier (max 7 points, i.e., +1)
  - Train a 7-class classifier = 6 basic emotions + neutral
  - Using your modified `train.tsv`

Create and evaluate a text classifier

- **Evaluation**: for each class, and overall (max 9 points, i.e., +1)
  - Using your modified **`test.tsv`**
  - Calculate and analyse a confusion matrix, precision/recall, F_1-score, average accuracy (micro vs. macro)
    - Compare your results to Table 6 in https://arxiv.org/pdf/2005.00547
  - Analyse your results, **e.g.**:
    - What is the impact of vocabulary pruning?
    - Does text clean-up/normalization or improved tokenization improve results?
    - Other observations, conclusions?
- **Extra task of your choice**: for a bonus point (max 10 points)
  - EITHER combine uni-grams and bi-grams in your vocabulary (feature vector) – e.g. `CountVectorizer`
    - AND compare to TF-IDF vectorization – e.g. TfidfVectorizer
  - OR train and use a BPE tokenizer (see e.g. TextPreprocessing.ipynb)
  - OR use another machine-learning approach (you might also need to use **`dev.tsv`**)
  - Compare these results to your best previous result
- **Mandatory requirements**:
  - Prepare a **concise report** about your work, experiments and choices, results and conclusions
  - **Self-declare** the tasks that you have completed; include important code snippets, visual materials
  - Submit a **self-contained single PDF** or **ipynb** file (filename and header contains author's full name)

Create and evaluate a text classifier

- **You are free to (re)use:**
  - Code from https://github.com/LUMII-AILab/NLP_Course/blob/main/notebooks/**NaiveBayes.ipynb**
    - See also **fastText.ipynb**, **BERT.ipynb**, **TextPreprocessing.ipynb**, etc.
  - Any standard or **third-party** Python library of your choice
  - Add essential comments (non-generated) to your code
  - Vibe coding is allowed, but you might be asked (at the final exam) to explain any part of your submission
    - i.e., you must understand everything you have submitted
  - Add references to the materials/resources/tools you have used